Welcome

Software Quality Engineering

Intro, Planning & Designing

Specialist Panel

Kapila Karunaratne

Experienced Delivery/QA head /Mentor/Lecturer/Researcher specialized in hiring, establishing, growing, and mentoring self-functioning offshore engineering teams for Software Development, Testing, and Consultancy with experience in setting up dedicated offshore teams fr key business clients such as IBM, LSEG, M&T, BMO, MEMX, and Virtusa's Global Automation Factory.

Holds a B.Sc. in Engineering, a B.Sc. (Hons) in Business Computing, and a Commonwealth Executive MBA. Also a certified Fintech Innovations Specialist, a Positive Psychology practitioner, and a Technology Leadership and Entrepreneurship graduate. IBM and ISTQB certified AI and QA specialist

Chathuranga Wijeratne

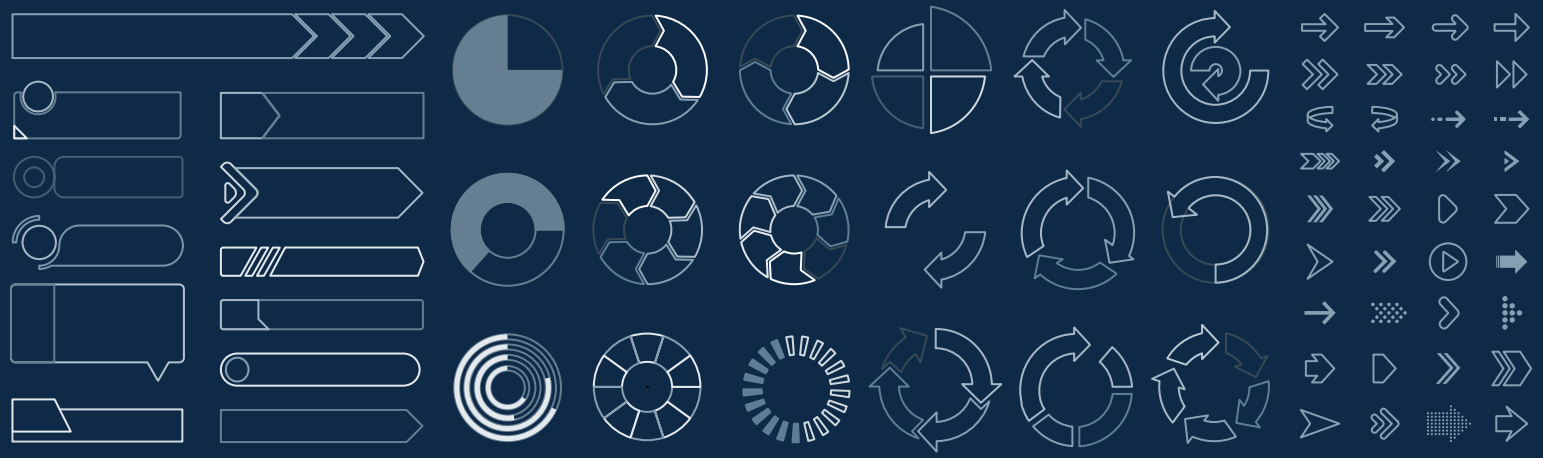
Experienced QA Manager on functional and Automation testing in MEMX, LSEG, BIT, OSLO, HNB, Sampath Bank and many capital market and Finance domain testing solutions .Holds B.Sc. in Applied Science, Science Faculty of the University of Peradeniya and ISTQB certified QA and AI Testing specialist.

Sarath Kadigamugedara

Experienced QA Manager on Non Functional & Automation testing in LSEG, JSE, NCDEX, ICX , HNB, Com Bank , Sampath Bank and many capital market and Finance domain testing solutions.

Holds B.Sc. in Computer Science and Technology, University of Uwa Wellassa and ISTQB certified QA and AI Testing specialist.

Quality Assurance Fundamentals



What is Testing ?

01 What is Software Testing?

02 What does it involve?

03 Why is Software Testing important?

What is software testing?

Software testing is not about making sure that everything is ok .

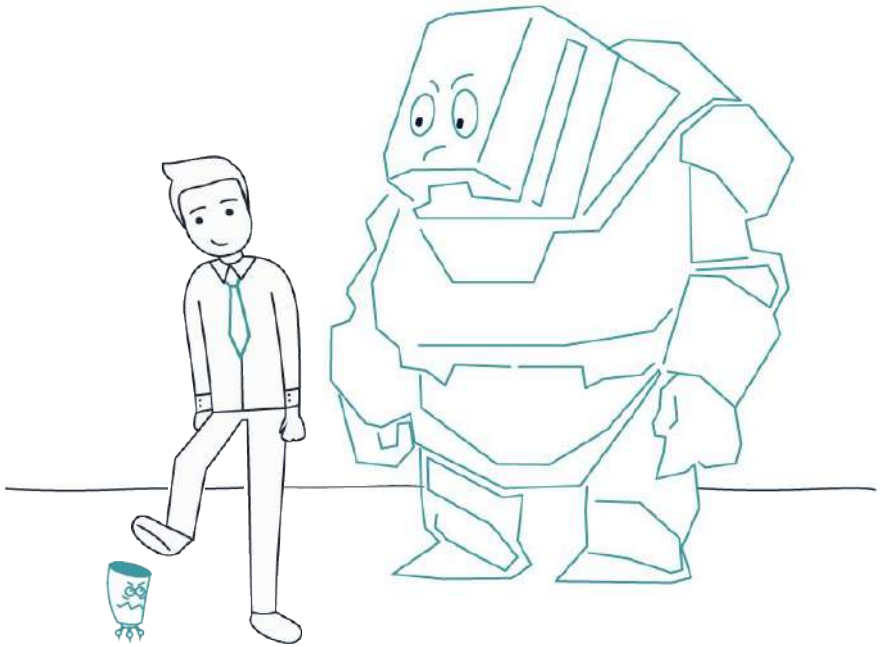
Software Testing is Relentless Learning

Software Testing is Relentless Learning

Software Testing is exploring software with the **intent** of finding bugs.

Software Testing is

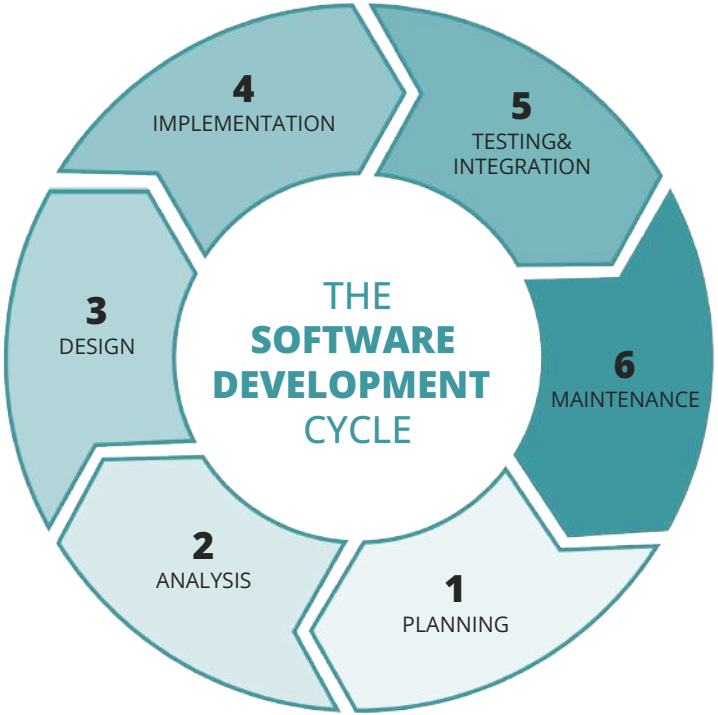
- an empirical
- technical
- investigation
- conducted to provide stakeholders
- with information
- about the quality
- of the product or service under test.



Why is Software Testing Important?

Software testing is an integral part of the software development life cycle because people are prone to making mistakes/errors. Some errors are insignificant, others may have costly repercussions.

Since **everyone makes mistakes**, we have to thoroughly verify the result of our work, as well as the work of others in everything we do.



Software Testing is of Utmost Importance

Ariane 5	Therac-25	Knight Capital Group	Video Assistant Referee
1996	1984	2012	2010
The destroyed rocket and its cargo were valued at $500 million	Radiation therapy machine caused injuries to 6 patients, 3 people died	Loss of $440 million, plus caused a major disruption of 148 companies pricing	VAR was first used in the Netherlands football championship without a test and caused a mess.

Software Testing is of Utmost Importance

Scientists Want To Shorten 1 Minute To 59 Seconds, As Earth's Spinning Faster

As per the data revealed by the scientists, the 24-hour daily rotation is decreasing considerably and reducing the time in our day.

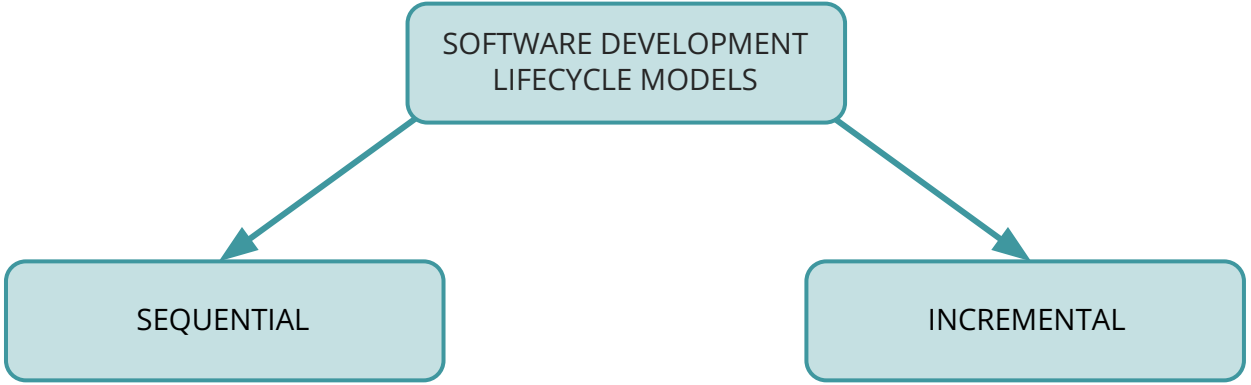
What is Software Development ?

01 What are Software Development Models?

02 What is Sequential development?

03 What is Incremental development?

Software Development Models

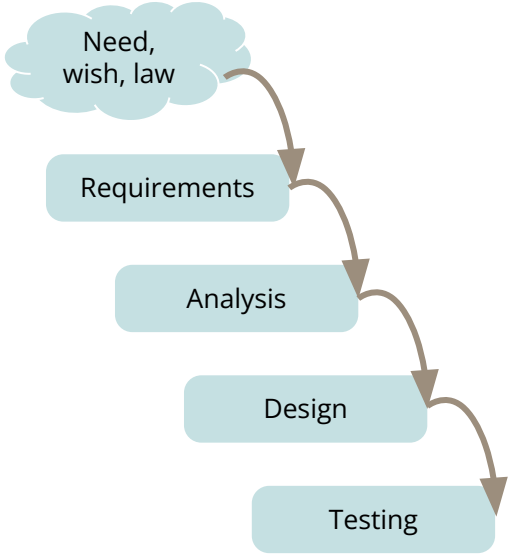


A **sequential development model** describes the software development process as a linear, sequential flow of activities. Any phase in the development process begins when the previous phase is complete.

Incremental development involves establishing requirements, designing, building, and testing a system in pieces, the software's features grow incrementally.

Sequential development models. The Waterfall Model

THE WATERFALL MODEL

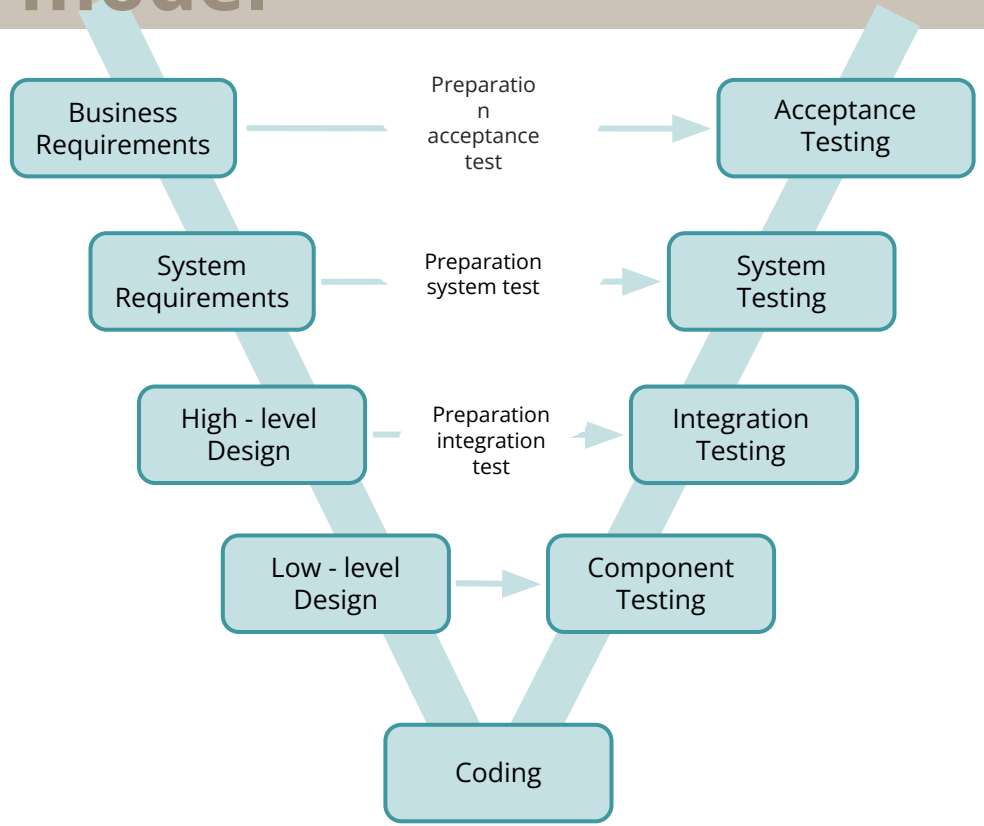


The waterfall model has a natural timeline where tasks are executed in a sequential fashion.

It is easy to manage and the process is well-documented.

It is hard to get a feedback passed up the waterfall. Test activities only occur after all other development activities have been completed.
High cost of defects.

Sequential development models. V-model



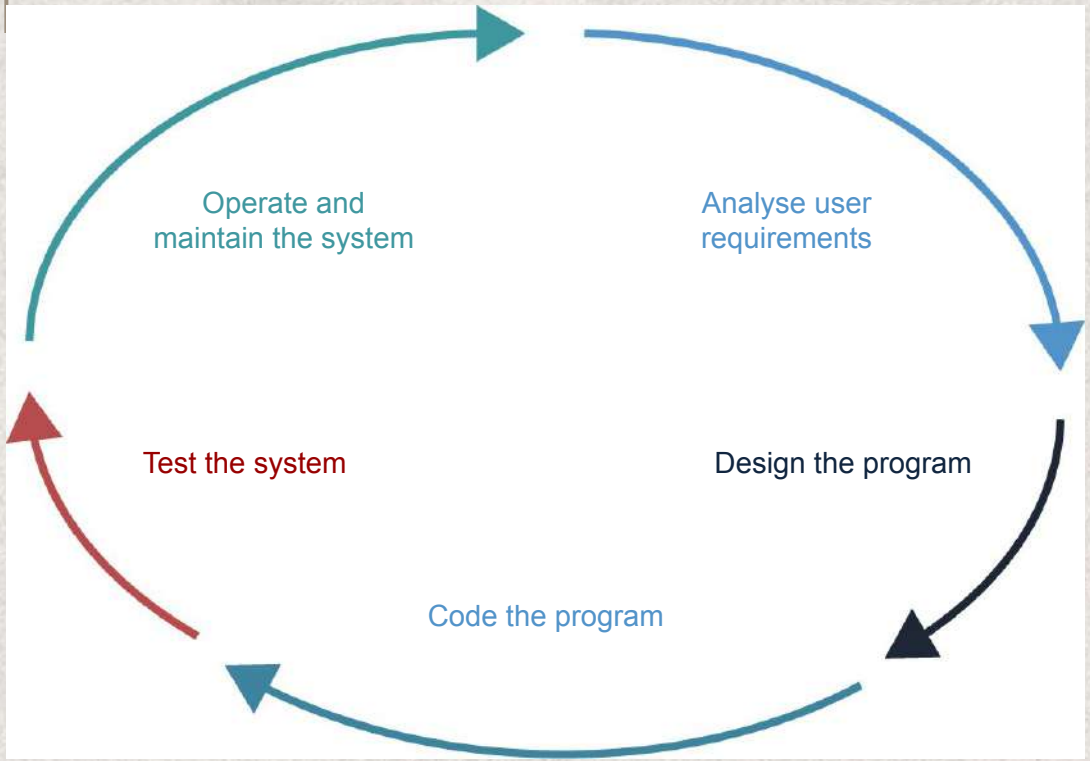
THE V-MODEL

The **V-model** associates test levels with corresponding development phases.

\+ The V-model provides guidance that testing needs to begin as ***early*** as possible.
Testing activities are carry out in ***parallel*** with development activities.

\- It still has limited built-in ability to respond to adjustment.
Testing is constrained by the rigid time frames.
There is a strict link between the corresponding level of development and testing.

Iterative development model

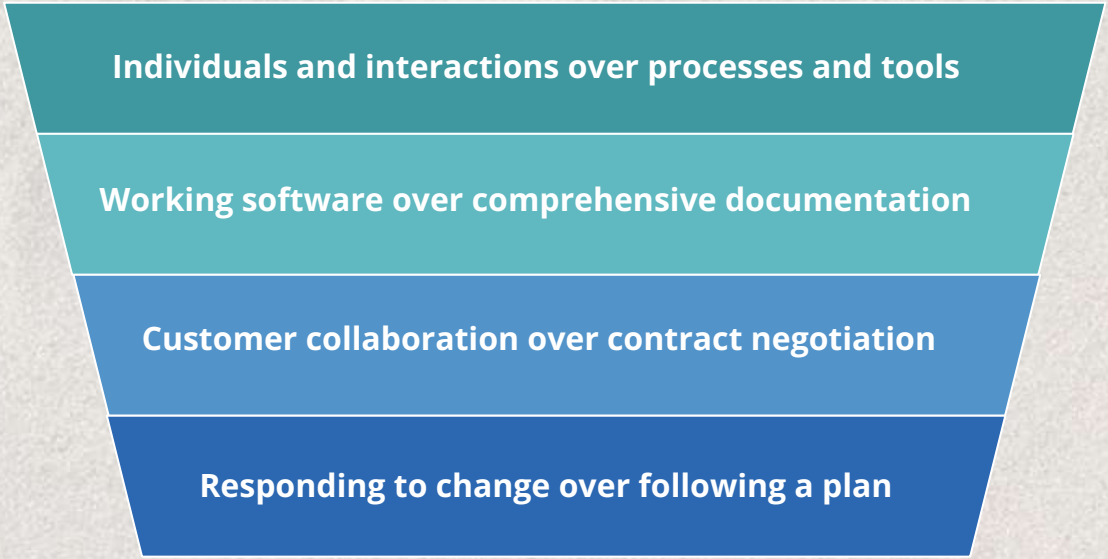


Iterative development occurs when groups of features are specified, designed, built, and tested together in a series of cycles, often of a fixed duration.

Regression testing is increasingly important on all iterations after the first one.

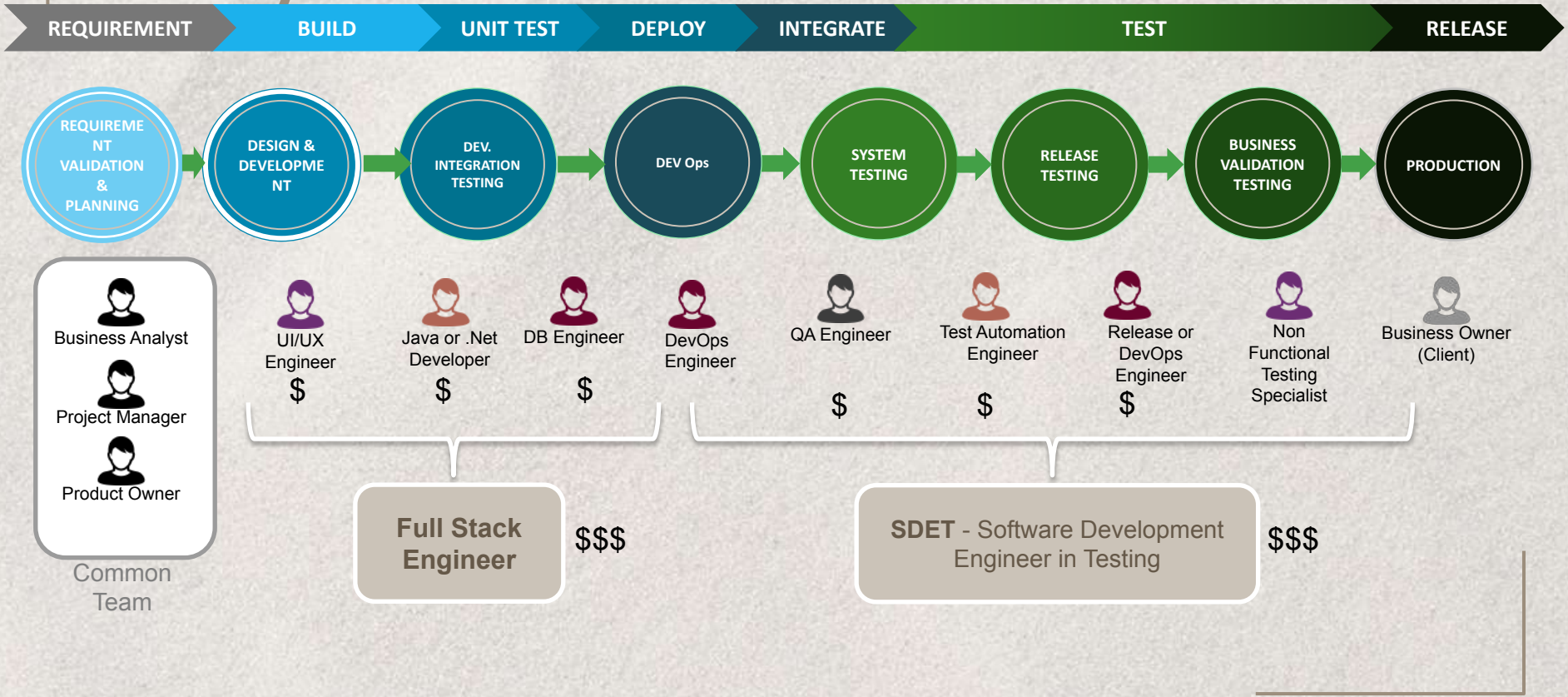
Agile development

The Manifesto for Agile Software Development



In 2001, these seventeen software developers met at a resort in Snowbird, Utah to discuss these lightweight development methods. Together they published the **Manifesto for Agile Software Development**.

Emerging Job Roles in IT Industry



What is Test Types ?

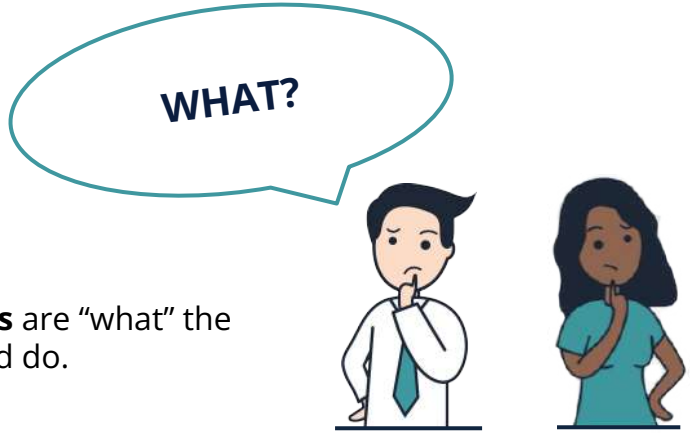
01 What are testing Types?

02 What is Product Risk?

03 What is Risk-based Testing?

Testing Types

functional testing



The **functions** are “what” the system should do.

non-functional testing

Non-functional testing is the testing of “how well” the system behaves.

Types of Software Testing

- Functional Testing
- Non-functional Testing
- Change-related Testing

Types of Testing

Functional

- Unit Testing
- Integration Testing
- Smoke/Sanity
- User Acceptance
- Localization
- Globalization
- Interoperability
- And so on...

Non-Functional

- Performance
- Endurance
- Load
- Volume
- Scalability
- Usability
- And so on...

Functional testing

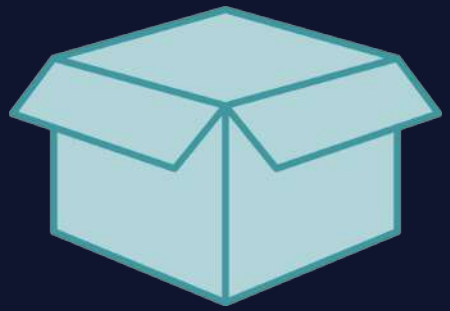
- Functional completeness
- Functional correctness
- Functional appropriateness

Testing functionality can be done from two perspectives:

- Requirements-based testing
- Experience-based testing
- Business-process-based testing

Non-functional Testing

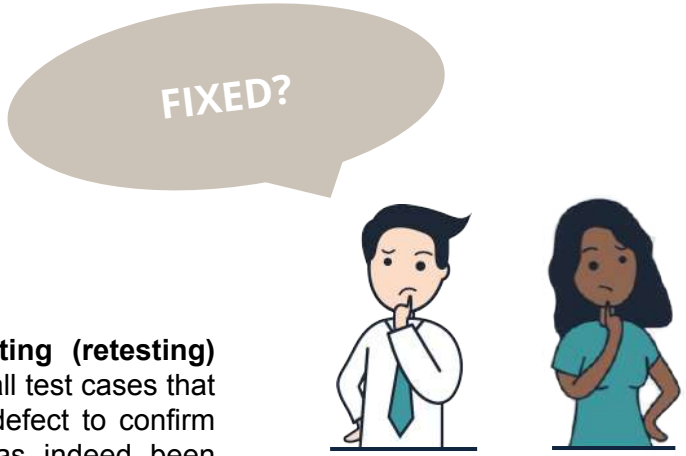
- Performance
- Load
- Stress
- Scalability
- Compatibility
- Usability
- Reliability
- Security
- Maintainability
- Portability



Change-Related Testing

Confirmation testing

Regression testing



Confirmation testing (retesting) is re-execution of all test cases that failed due to the defect to confirm that the defect has indeed been fixed.

The purpose of **regression testing** is to verify that modifications in the software have not caused unintended adverse side effects and that the system still meets its requirements.

Product and Project Risks

PRODUCT

- Software cannot perform its intended functions.
- A system architecture may not adequately support some non-functional requirements.
- A particular computation may be performed incorrectly in some circumstances.
- A loop control structure may be coded incorrectly.
- Response-times may be inadequate for a high-performance transaction processing system.
- User experience feedback might not meet product expectations.

PROJECT

- Organizational issue
- Project issues
- Political issues
- Technical issues
- Supplier issues

Risk management

Analyze what can go wrong	Determine the priority of risks
Implement actions to mitigate those risks	Make contingency plans

Risk-based Testing and Product Quality

To determine

- the test techniques to be used.
- the particular levels and types of testing to be performed.
- the extent of testing to be carried out for the different levels and types of testing.
- whether any activities in addition to testing could be employed to reduce risk.

To prioritize testing

Group Exercise

Case Study

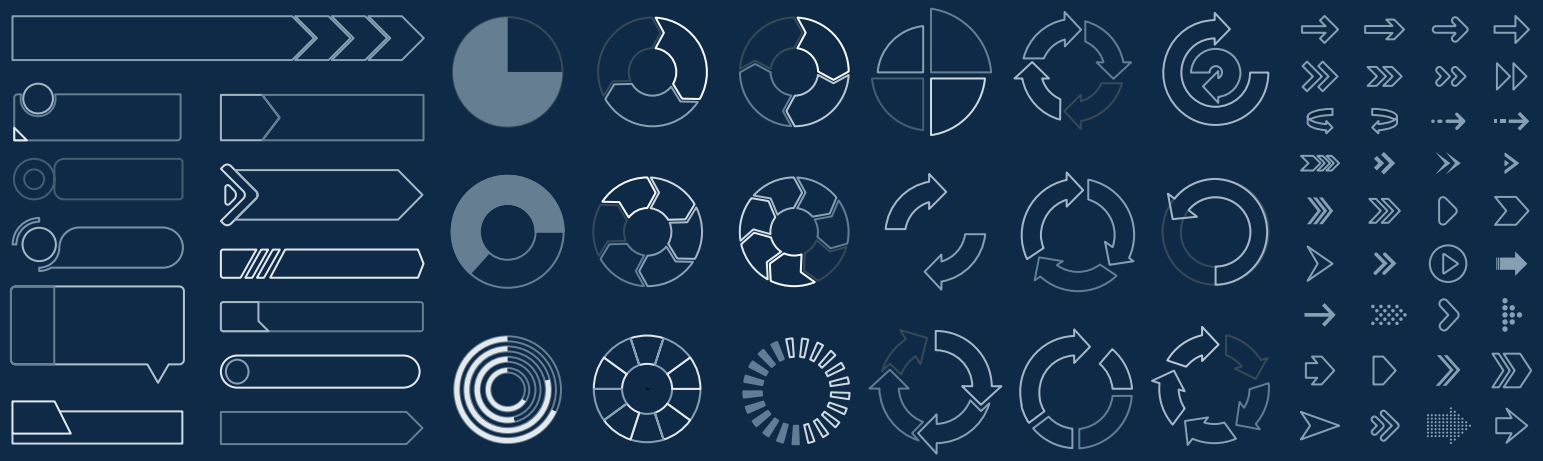
Testing Types for Taxi Booking Application

The Internet and smartphone era has brought changes to the well-known offline services and has influenced every corner of our everyday life. If someone needs to book a flight, train tickets, a room in a hotel or order something in a restaurant, they most likely will use a mobile app for these purposes.

Taxi services are certainly no exception to this. The traditional way to book a taxi is by physically waving one down in the street or by calling the taxi company. These methods are hardly used anymore. Booking services such as Pick Me that use mobile apps for their business are currently valued most in the taxi market in Sri Lanka.

- **Understand the Business requirement of a Taxi Booking**
- **Identify 5 software risks and mitigation plans for testing for above requirement**

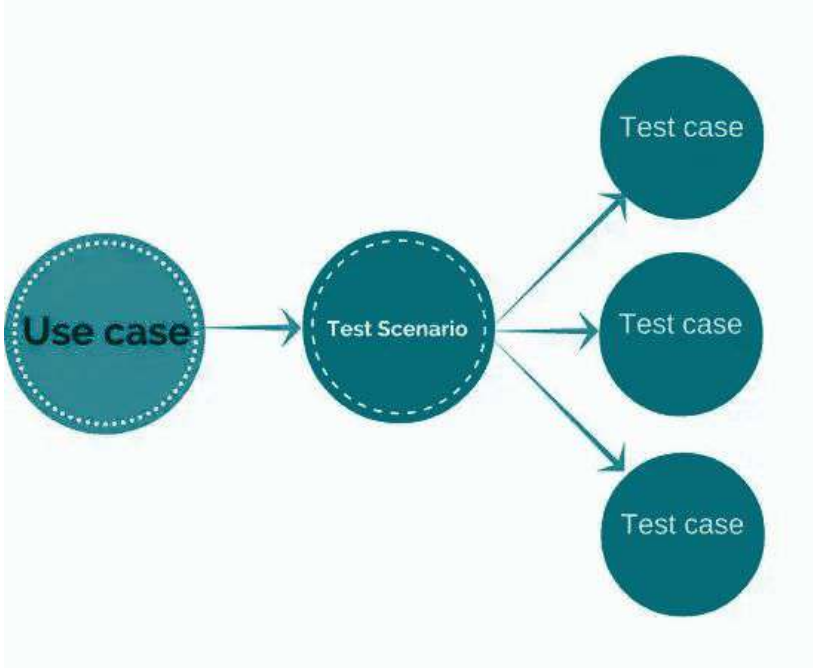
What are Test Cases ?



What is a Test Case & Test Case Designing

- A test case is a document that describes an input, action or event and an expected response, which is used to determine if a feature of an application is working correctly

- The key issue under Test case designing is to identify a subset of all possible Test Cases (set of data to be used as input) that have a high probability of detecting errors



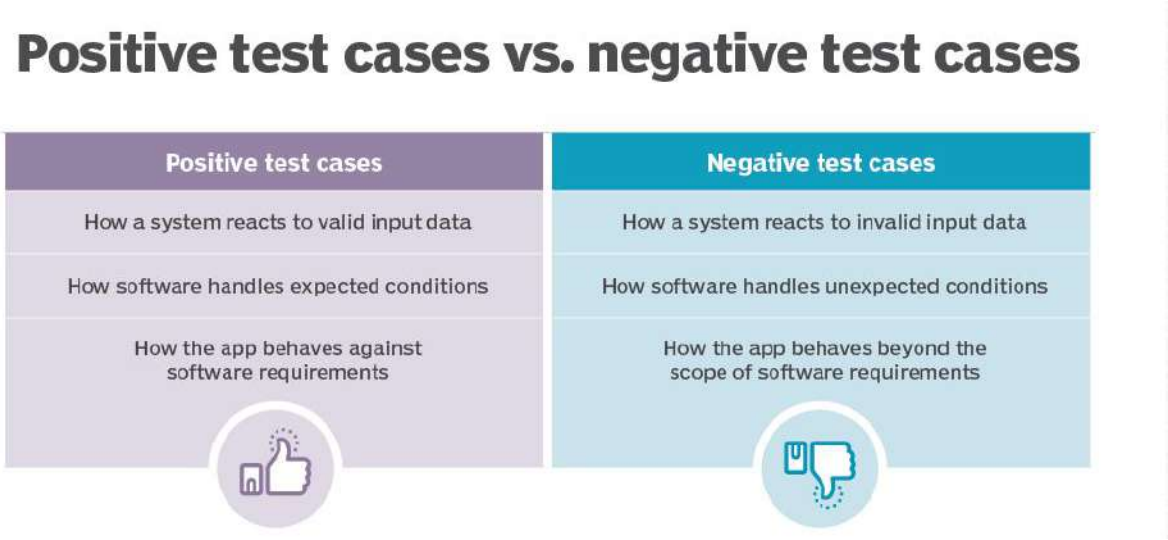
Why We Need Test Cases?

- Provides a standardized set of baselines for each testing cycle
- Help “jump start” the testing for the inexperienced testers or for new team members
- Help define the scope of testing – Stop asking “Did I miss anything?”
- Repeatability – Help Repeat the same tests regardless of the tester
- Help correctly estimate the effort for future regression test cycles

Types of Test Cases

- Positive Test Cases: Entering valid data into an application to determine if its processing results are correctly produced as defined in the requirement specification.

- Negative Test Cases: Entering incorrect data intentionally to determine if an application will recognize the data entered is invalid and take appropriate action



Elements of a Test Case

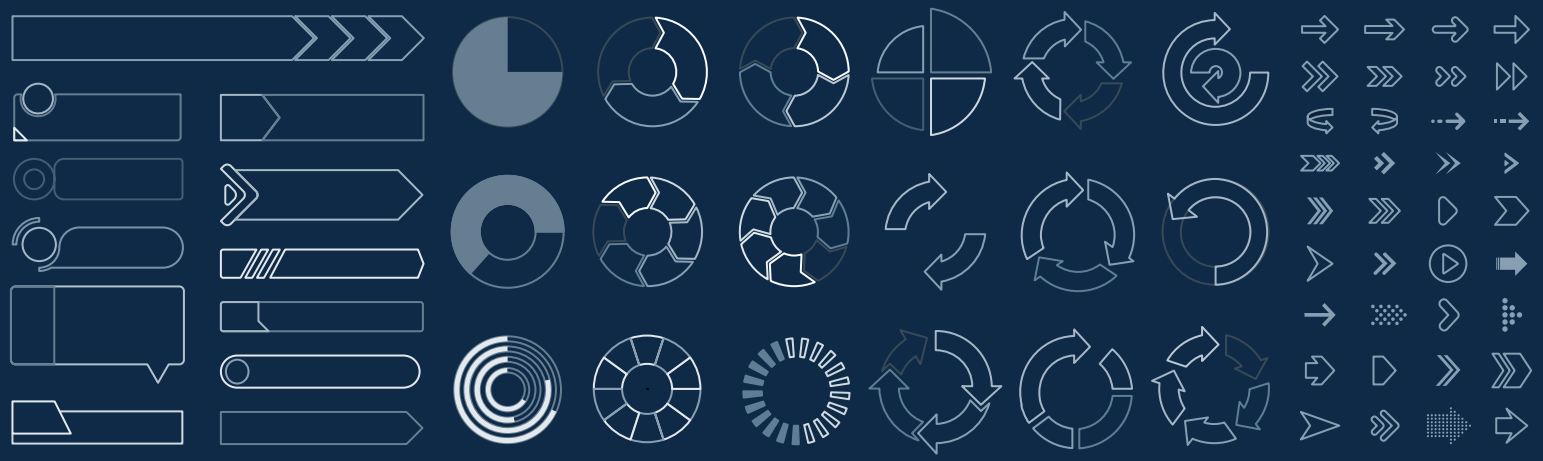
- Test Case ID
- Test Case Objective/Test Case Description
- Pre Conditions
- Test Input data
- Test Steps
- Test Expected Results
- Test Actual Results

Test ID	Test Objective	Preconditions	Steps	Expected results	Actual results
#1	Email address is a mandatory field in Login page	Browser is opened User is in the Login Page	1) User enter an Email address to the username	Green colored tick should be displayed	

Attributes of a Good Test Case

- The Test Case objective should be Brief & Clear
- The Test Case should correctly Verify the requirement
- Expected result should specify the Behavior of the function being tested when we do certain action
- Test Case should be Easy to determine the result
- Test Case should be Maintainable and Mangable

Test design techniques



What is a test design technique?

- A procedure for selecting or designing tests
- Based on a structural or functional model of the software
- Successful at finding faults
- ‘Best' practice
- A way of deriving good test cases

Test Design Techniques and Test Execution

Topics to Cover !

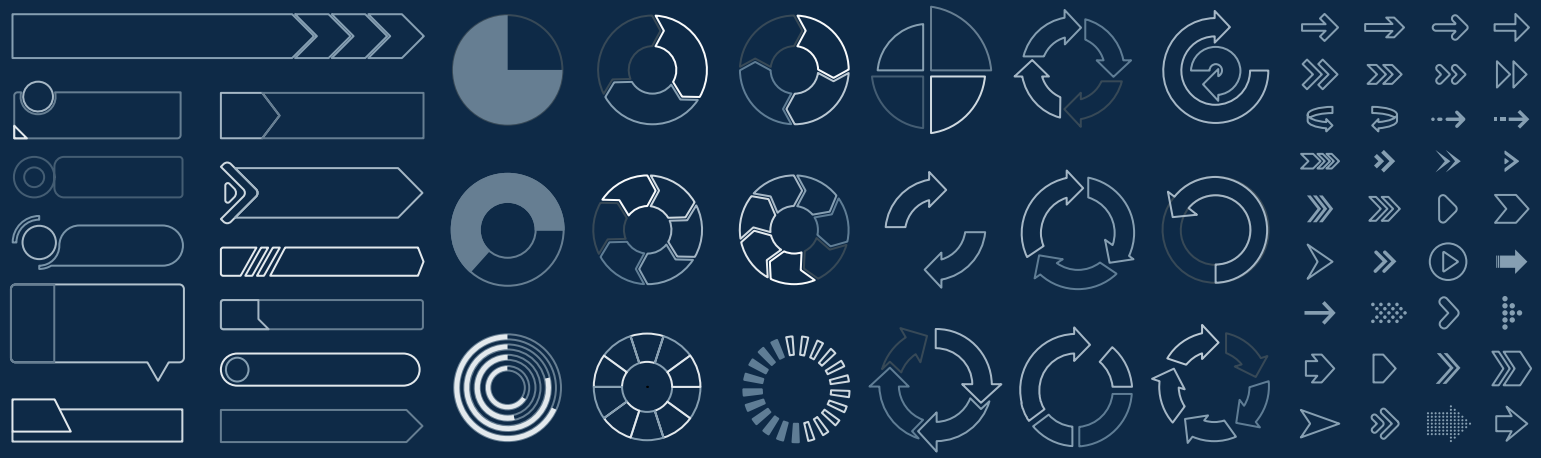
01 What are Test Cases

02 Test design techniques

03 Categories of Tests

04 Black Box Testing Techniques

Categories of Tests



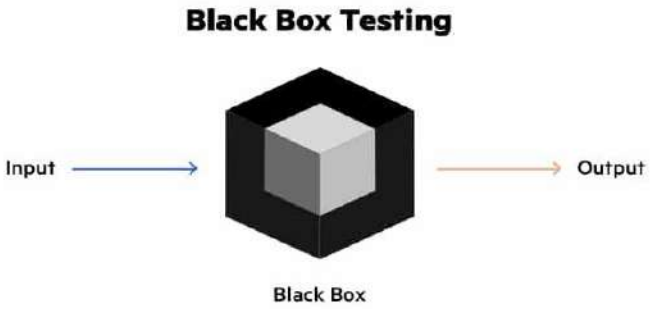
Categories of Test Design Techniques

- The Test Design Techniques can be widely categorized as:
  - Specification-Based or Black Box Design Techniques
  - Structure based or White Box Design Techniques
  - Gray(Grey) Box Testing(Hybrid of Black Box and White Box)
  - Experienced Based Testing Techniques

Black box testing (also called functional testing)

- Black Box Testing, also known as Behavioral Testing, is a software testing method in which the internal structure/design/implementation of the item being tested is not known to the tester. These tests can be functional or non-functional, though usually functional.

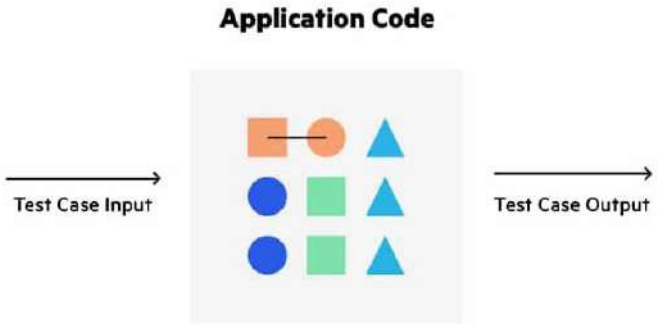
- Black Box Testing method is named so because the software program, in the eyes of the tester, is like a black box; inside which one cannot see.



White box testing (also called structural testing)

- White box testing deals with the internal logic and structure of the code. White box testing is also called as glass, structural, open box or clear box testing. The tests written based on the white box testing strategy incorporate coverage of the code written, branches, paths, statements and internal logic of the code etc.

- In order to implement white box testing, the tester has to deal with the code and hence is needed to possess knowledge of coding and logic i.e. internal working of the code. White box test also needs the tester to look into the code and find out which unit/statement/chunk of the code is malfunctioning.



Grey Box Testing(Hybrid of Black Box and White Box)

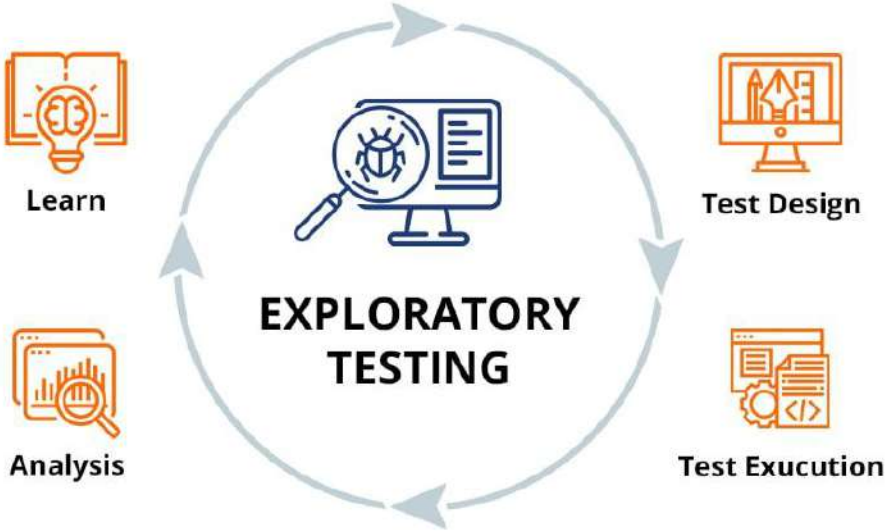
- Grey Box Testing is a software testing method which is a combination of Black Box Testing method and White Box Testing method.

- In Grey Box Testing, the internal structure is partially known. This involves having access to internal data structures and algorithms for purposes of designing the test cases, but testing at the user, or black-box level.

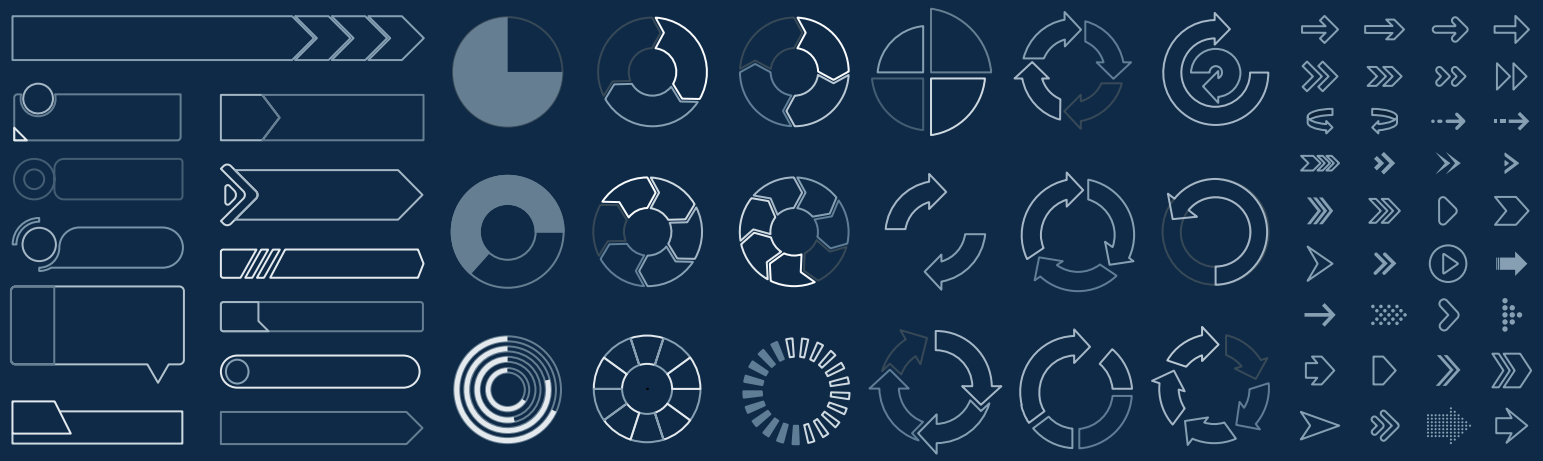
- Grey Box Testing is named so because the software program, in the eyes of the tester is like a gray/semi-transparent box; inside which one can partially see

Experience-Based Testing

- Exploratory testing: concurrent test design, test execution, test logging and learning are carried out within time-boxes. It is an approach that is most useful where there are few or inadequate specifications and severe time pressure, or in order to augment or complement other, more formal testing.



Black Box Testing Techniques



Black Box Test Design Techniques

- **Equivalence partitioning**
  Dividing input values into valid and invalid partitions and selecting representative values from each partition as test data.

- **Boundary Value Analysis**
  Determination of boundaries for input values and selecting values that are at the boundaries and just inside/outside of the boundaries as test data.

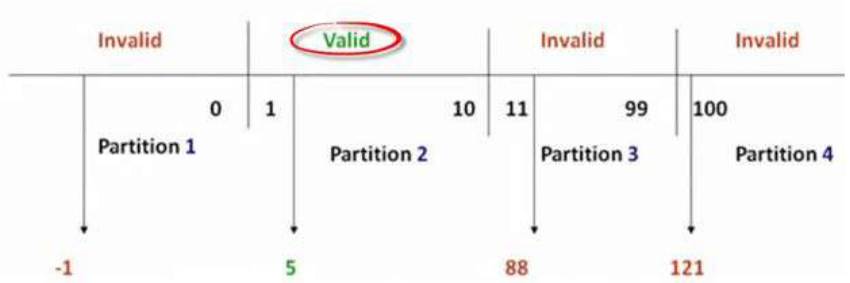
- **State Transition Testing techniques**
  State transition diagram indicates how the system behaves as a consequence of external events and assess the program accuracy by analyzing the conditions under which a state change occurs.

- **Decision Table**
  Testing technique used to test system behavior for different input combinations

- **Error Guessing**
  Tests are derived from the tester’s skill and intuition and their experience with similar applications and technologies.

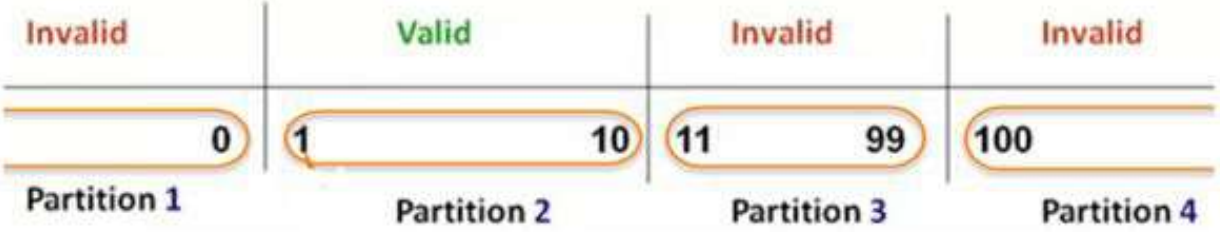
Equivalence Partitioning

- Equivalence partitioning is the process of taking all of the possible test values and placing them into classes (groups).
- Each element of a class should be “equivalent” in its likeliness to expose a bug.
- By using equivalence partitioning, we only need to use a few test cases per class to represent all test values.
- For example, for a input which accept integers between 1 and 10, the following partitions exist.
    - Input values which less than 1
    - Input values between 1 and 10
    - Input values greater than 10



Boundary-Value Analysis

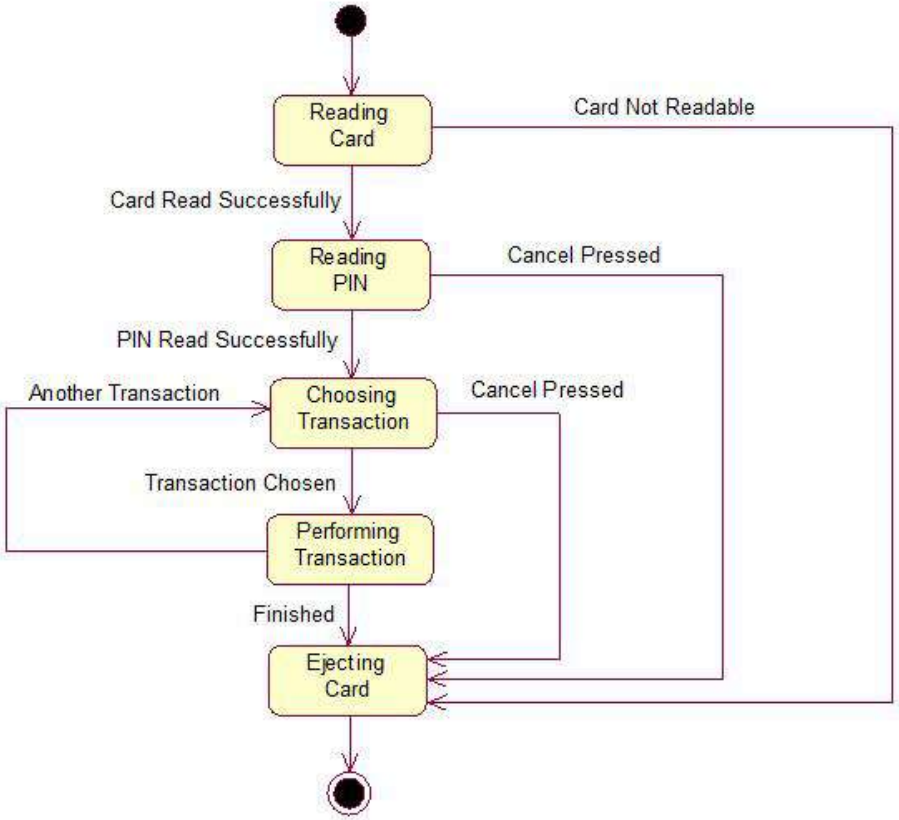
- A technique based on identifying, and generating test cases to explore boundary conditions
- Boundary conditions are an extremely rich source of errors
- Select test cases that lie directly on, above, and beneath the boundaries of input equivalence classes and output equivalence classes to explore the program behavior along the border
- Boundary value analysis may be termed as an extension/Application of equivalence partition technique



State Transition Testing Technique

- Identify a finite number of states the program execution goes through.
- Create a state transition diagram showing how the program transitions from one state to the other.
- Assess the program accuracy by analyzing the conditions under which a state change occurs.
- State transition diagram indicates how the system behaves as a consequence of external events.

State Transition Testing Technique



Decision Table

- Systematic approach where the different input combinations and their corresponding system behavior (Output) are captured in a tabular form. That is why it is also called as a Cause-Effect table where Cause and effects are captured for better test coverage.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

- Case 1 – Username and password both were wrong. The user is shown an error message.
- Case 2 – Username was correct, but the password was wrong. The user is shown an error message.
- Case 3 – Username was wrong, but the password was correct. The user is shown an error message.
- Case 4 – Username and password both were correct, and the user navigated to homepage

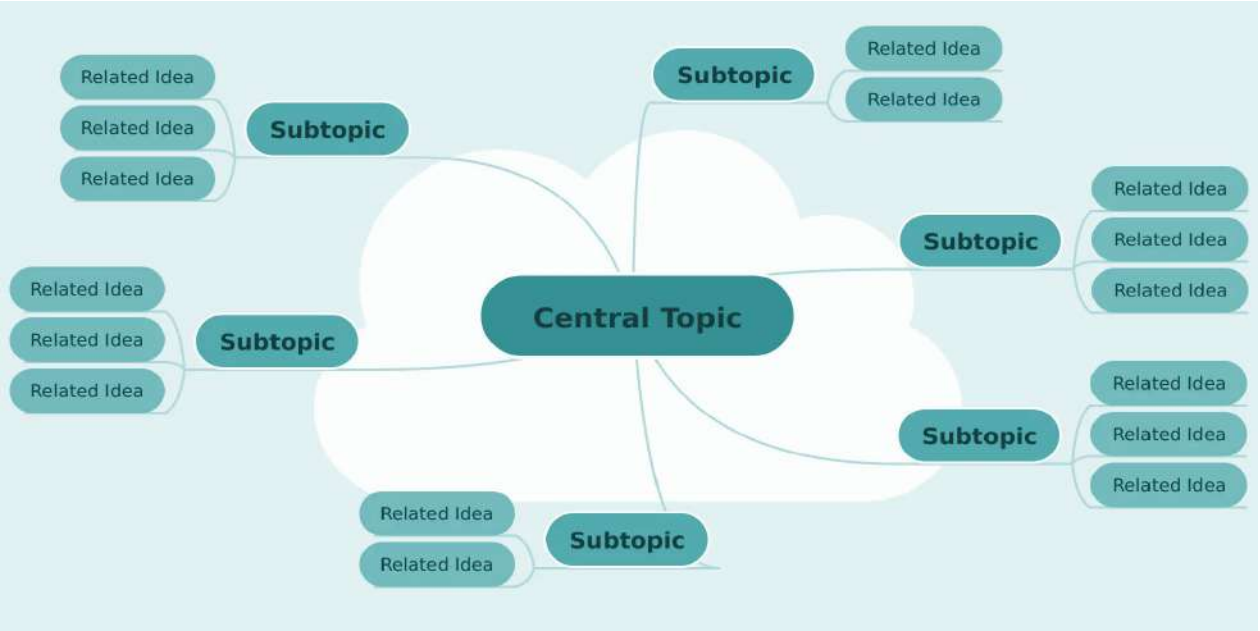
Error Guessing

- Effective testing technique based on experience of the tester
- Just ‘guess’ where the errors are ……
- Intuition and experience of tester
- Ad hoc, not really a technique
- But can be quite effective
- Construct a list by identify all possible errors or error-prone situations, and then use it to select test cases

Test Designing with Mind Maps

Challenges in Traditional Test Designing methods

- Used word or excel based documents to generate scenarios
- Difficult to track the traceability in large project
- Difficult to get the whole picture of the system

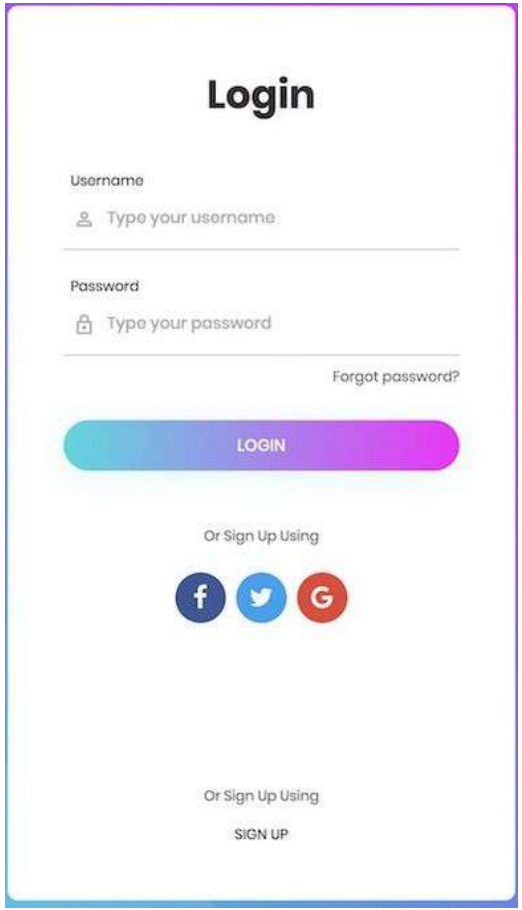


Advantages of MindMaps

- Can identify the data/transaction flows of the system clearly.
- Traceability is clearer
- Easy to maintain
- Easy to understand
- Easy to search

Sample Scenario

- Business Requirement
    - Username is mandatory
    - Username accepts only email addresses
    - Password is mandatory
    - Password can have 8-16 characters
    - When user click on Login button user should be logged into the Homepage of the application
    - If wrong Credentials are provided error message should be given as “Invalid Credentials.”



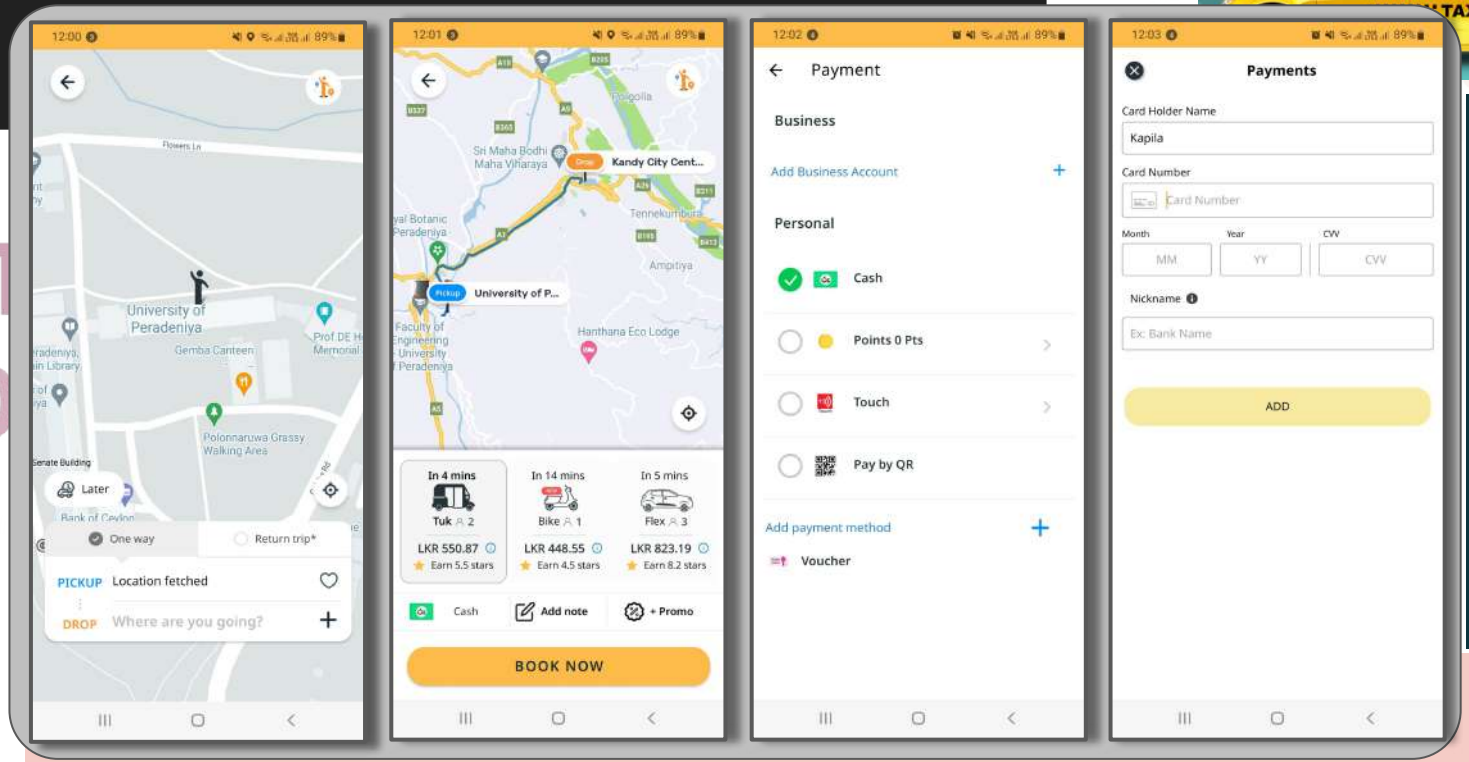
Sample Test Cases

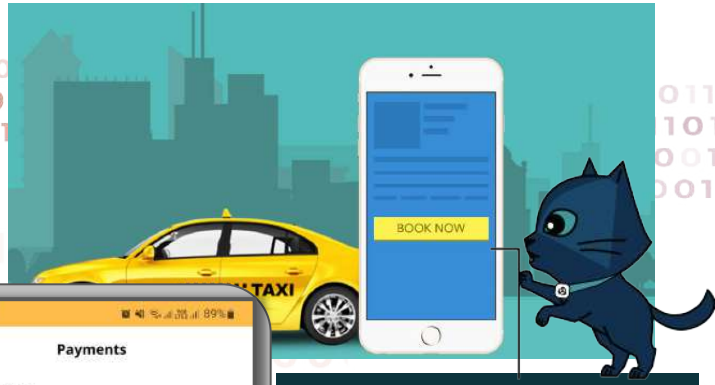
TC ID	Test Objective	Pre Conditions	Steps	Expected Results	Actual Results	Testing Technique Used
1	User should be able to login with valid credentials	User is in the login page of the application	1. User enter a valid username 2. User enters a valid password 3. User click on the login button	User should be logged into the application		**Decision Table**
2	User can login only with valid credentials	User is in the login page of the application	1. User enter a valid username 2. User enters a incorrect password 3. User click on the login button	Error message should be given as “Invalid credentials”		**Decision Table**
3	Password should have 8-16 characters	User is in the login page of the application	1. User enters a valid username 2. User enters a password with length 10 3. User click on the login button	User should be logged into the application		**Equivalence Partition**
4	Password should have 8-16 characters	User is in the login page of the application	1. User enters a valid username 2. User enters a password with length 7 3. User click on the login button	User should not be able to login into the application		**Boundary Value Analysis**
5	User should be navigated to homepage once logged in	User is in the login page of the application	1. User enter a valid username 2. User enters a valid password 3. User click on the login button	User should be in Homepage of the application		**State transition**

Group Exercise

✔ Case Study

Test Design for Taxi Booking Application



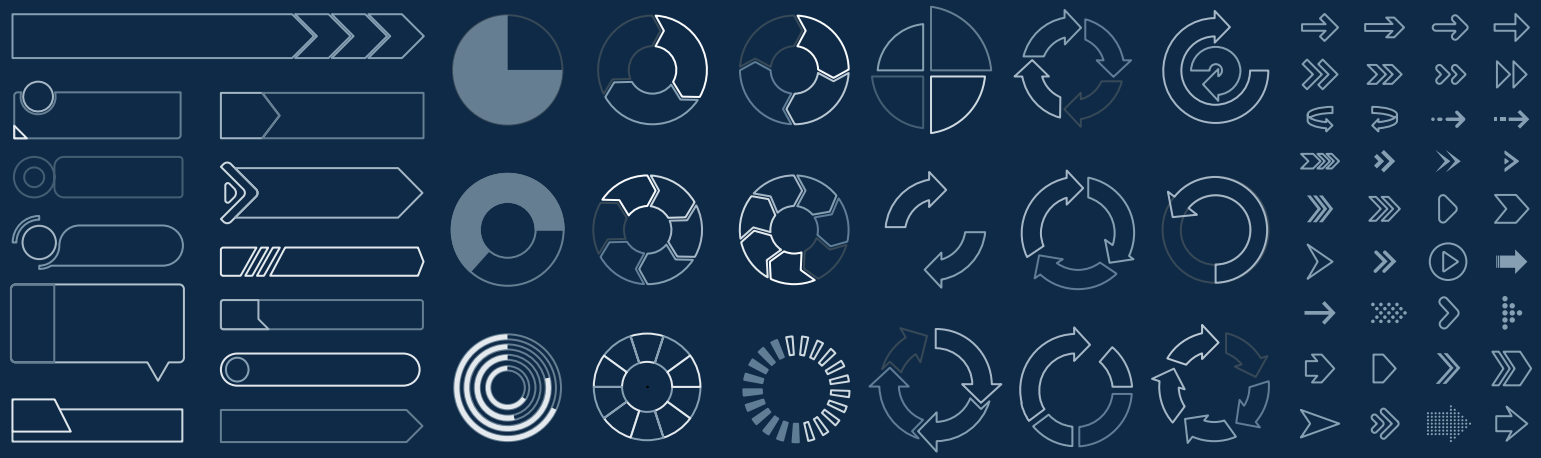


Develop 10 Test cases covering below test design Techniques

- Equivalence partitioning
- Boundary Value Analysis
- State Transition Testing techniques
- Decision Table
- Error Guessing

exactpro
EXITUS ACTA PROBAT

Test Execution



Test Execution

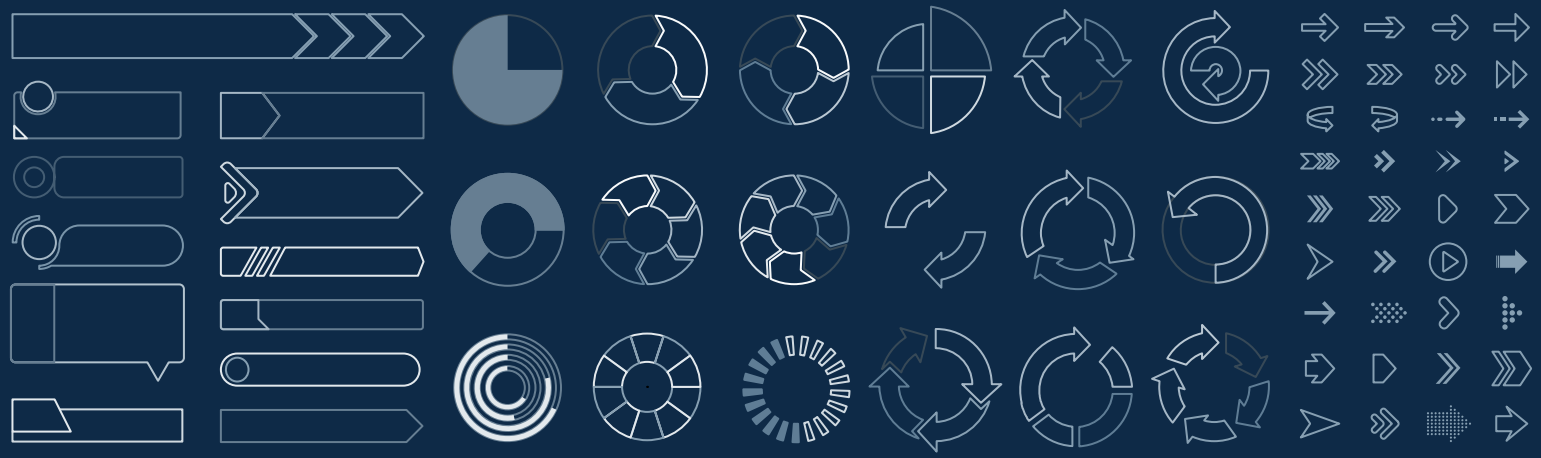
There can be two types of executions

Manual

Automation

Test Execution Tools

Software Defects



Bugs or Defects

- What is a Bug
- Can we have zero bug system
- How to identify a bug
- How to report a Bug
- Bug Life cycle

How to determine a bug and how to report a bug

TC ID	Test Objective	Pre Conditions	Steps	Expected Results	Actual Results
1	User can login only with valid credentials	User is in the login page of the application	1. User enter a valid username 2. User enters a incorrect password 3. User click on the login button	Error message should be given as “I**nvalid Credentials.**”	User logged in to the application

- If there is a deviation between the Actual and Expected results we can assume that there is a bug.
- If we are confident that the test case is correct and the test conditions are correct we can raise a bug
- If we have any doubts we can go through the requirements and confirm the behavior
- Every failed TC should have a Bug ID

Important points to remember when raising a bug

- Bug should have a proper description/title
- Informative description with steps, environment details, Test Data
- Expected and actual results
- Any references of the expected system behavior
- Severity & Priority
- Relevant screenshots/videos/logs

Sample Bug Report

FX Surveillance / FS-21918

[MRS] Empty event descriptions displayed for session changes in MRS event navigator

Edit | Add comment | Assign | More | Pending | Resolve Issue | Close Issue | Export

Details

Type:	Bug	Status:	OPEN
Priority:	Medium	Resolution:	Unresolved
Affects Version/s:	1.0.8.0.i2	Fix Version/s:	None
Component/s:	None		
Labels:	FX_CORE_TRIAGED NDF_Phase1		

Field Tab | MIT Details

Severity Level:	MEDIUM
Product/s:	FX Core Surveillance
Site:	N/A
Environment Type:	PST ST
Environment Name:	ST1
Test Phase:	PST ST
Test Type:	FT
Defect Source Type:	Regression Defect

Description

Scenario:

Create a session for an instrument in MRS and observe the events in the Event navigator

Expected results:

All the events should contain proper event descriptions

Actual results:

Event descriptions are displayed as Empty. As per the time stamps of the events the empty events can be mapped to the session change requests in Yaala.

People

Assignee:	Veronica Kuverova
	Assign to me
Reporter:	Chathuranga Wijerathne
Votes:	0
Watchers:	9 Stop watching this issue

Dates

Created:	21/Dec/22 1:09 PM
Updated:	20/Feb/23 3:05 PM

Development

Create branch

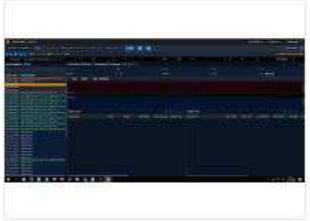
Agile

View on Board

Sample Bug Report

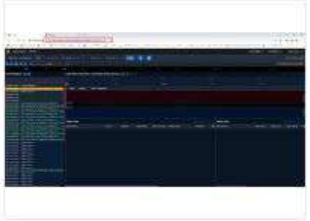
Attachments

Drop files to attach, or browse.



Empty Event Description.PN
21/Dec/22 1:08 PM 188 kB

Empty Event Description Dr
21/Dec/22 3:07 PM 220 kB



Empty Event Description Dr
21/Dec/22 3:07 PM 188 kB

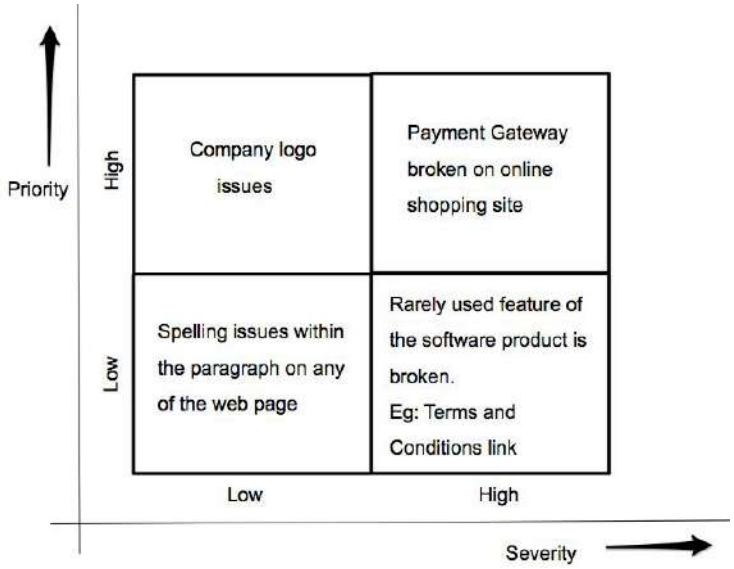
MarketManagerActivityLog.
21/Dec/22 1:09 PM 9 kB

MarketReplay_1_1_EventSer
21/Dec/22 3:12 PM 3.67 MB

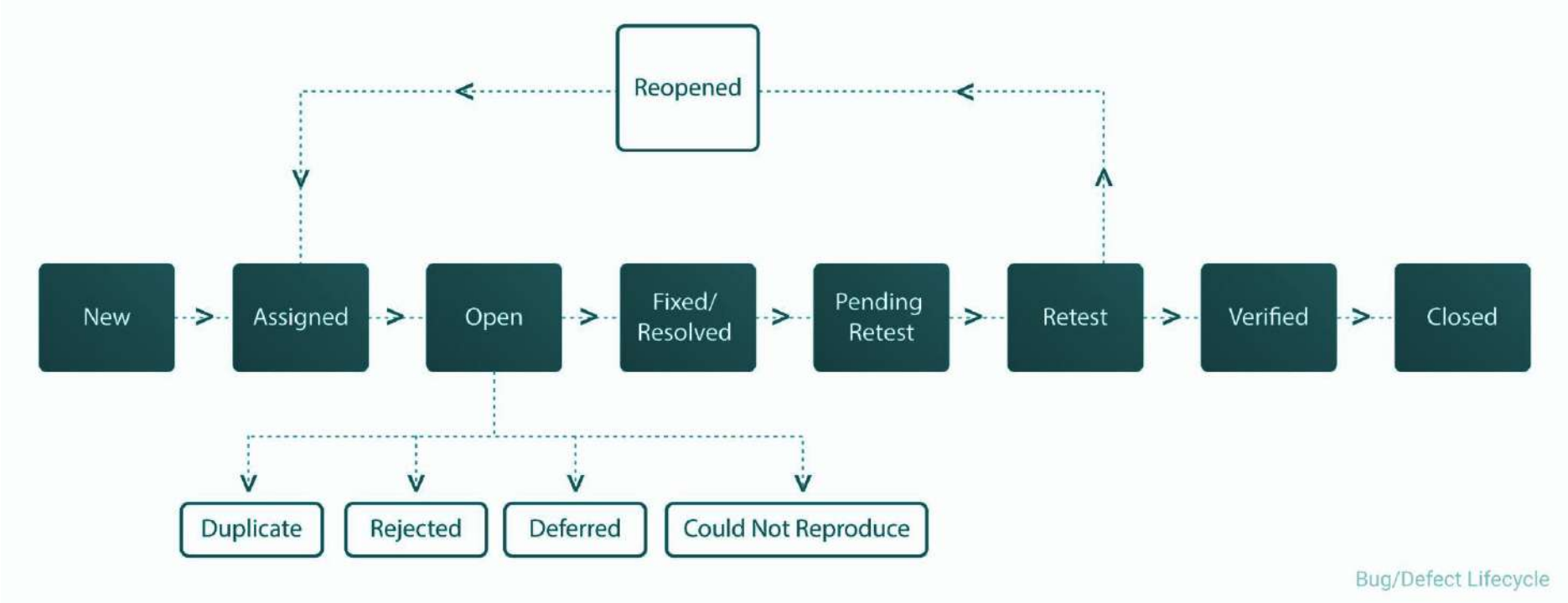
Severity & Priority

- **Severity** - How much the bug is impacting the system functionality
  - **Low**: Bug won't result in any noticeable breakdown of the system
  - **Minor**: Results in some unexpected or undesired behavior, but not enough to disrupt system function
  - **Major**: Bug capable of collapsing large parts of the system
  - **Critical**: Bug capable of triggering complete system shutdown

- Priority - How quickly should the bug needs to be addressed.
  - **Low**: Bug can be fixed at a later date. Other, more serious bugs take priority
  - **Medium**: Bug can be fixed in the normal course of development and testing.
  - **High**: Bug must be resolved at the earliest as it affects the system adversely and renders it unusable until it is resolved.



Defect Life Cycle



Defect Management Tools

How to determine a bug and how to report a bug

TC ID	Test Objective	Pre Conditions	Steps	Expected Results	Actual Results
1	User can login only with valid credentials	User is in the login page of the application	1. User enter a valid username 2. User enters a incorrect password 3. User click on the login button	Error message should be given as “I**nvalid Credentials.**”	User logged in to the application

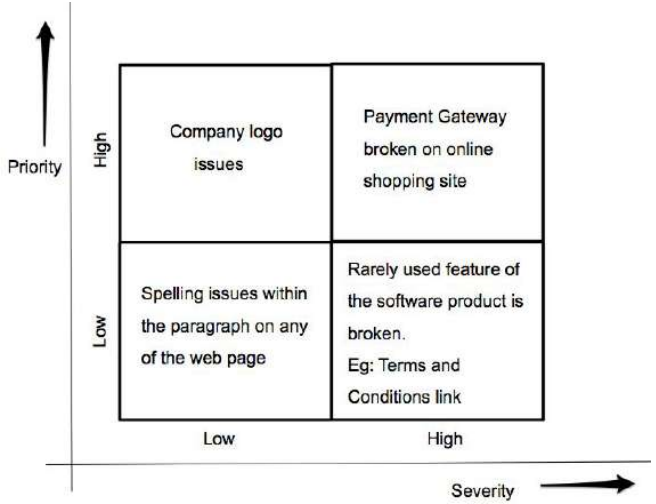
- If there is a deviation between the Actual and Expected results we can assume that there is a bug.
- If we are confident that the test case is correct and the test conditions are correct we can raise a bug
- If we have any doubts we can go through the requirements and confirm the behavior
- Every failed TC should have a Bug ID

Important points to remember when raising a bug

- Bug should have a proper description/title
- Informative description with steps, environment details, Test Data
- Expected and actual results
- Any references of the expected system behavior
- Severity & Priority
- Relevant screenshots/videos/logs

Severity & Priority

- **Severity** - How much the bug is impacting the system functionality
  - **Low**: Bug won't result in any noticeable breakdown of the system
  - **Minor**: Results in some unexpected or undesired behavior, but not enough to disrupt system function
  - **Major**: Bug capable of collapsing large parts of the system
  - **Critical**: Bug capable of triggering complete system shutdown
- Priority - How quickly should the bug needs to be addressed.
  - **Low**: Bug can be fixed at a later date. Other, more serious bugs take priority
  - **Medium**: Bug can be fixed in the normal course of development and testing.
  - **High**: Bug must be resolved at the earliest as it affects the system adversely and renders it unusable until it is resolved.



Questions ?

Quiz Time

Thanks !

kapilalk@gmail.com
+94 71 664 3373
https://www.linkedin.com/in/kapila-karunaratne-a3783926